

Employee Authentication Database Context

The **AuthDbContext** class configures the authentication database context for the Employee Management API. It inherits from **IdentityDbContext<IdentityUser>**, which provides the Entity Framework Core integration needed to store and manage user identity data.

- **Identity support:** Uses ASP.NET Core Identity to manage user accounts and authentication data.
- **Entity Framework Core integration:** Accepts **DbContextOptions<AuthDbContext>** through the constructor so the database provider can be configured in application startup.
- **Clean inheritance model:** Extends **IdentityDbContext<IdentityUser>** to reuse the built-in Identity schema and behavior.

The following code defines the authentication database context:

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace EmployeeManagementAPI.Data
{
    public class AuthDbContext : IdentityDbContext<IdentityUser>
    {
        public AuthDbContext(DbContextOptions<AuthDbContext> options)
            : base(options)
        {
        }
    }
}
```

Application Startup and Service Configuration

This section configures the Employee Management API startup pipeline. It registers the required services, enables authentication-related identity endpoints, configures

development CORS access, and sets up middleware for Swagger, HTTPS redirection, authorization, controllers, and application execution.

- **Database context:** Registers **AuthDbContext** and uses an in-memory database named **AuthDb** for authentication data.
- **Authorization and Identity:** Adds authorization services and maps ASP.NET Core Identity API endpoints for **IdentityUser**.
- **CORS policy:** Creates an **AllowDev** policy that allows any origin, header, and method for development use.
- **API and documentation support:** Adds controllers, endpoint API exploration, and Swagger generation.
- **Request pipeline:** Enables CORS, Swagger in development, HTTPS redirection, authorization, controller routing, and application startup.

The following code shows the complete startup configuration:

```
using EmployeeManagementAPI.Data;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Register services.
builder.Services.AddDbContext<AuthDbContext>(options =>
    options.UseInMemoryDatabase("AuthDb"));

builder.Services.AddAuthorization();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowDev", policy =>
        policy.AllowAnyOrigin()
            .AllowAnyHeader()
            .AllowAnyMethod());
});

builder.Services.AddIdentityApiEndpoints<IdentityUser>()
```

```
.AddEntityFrameworkStores<AuthDbContext>();

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

app.UseCors("AllowDev");
app.MapIdentityApi<IdentityUser>();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

LINQ and Lambda Expressions

LINQ (Language Integrated Query) and **lambda expressions** are commonly used in C# to query, filter, transform, and work with collections or database data in a concise and readable way.

- **LINQ:** Provides a consistent query syntax for working with arrays, lists, objects, and database results.
- **Lambda expressions:** Define short inline functions, often used with LINQ methods such as **Where**, **Select**, **OrderBy**, and **FirstOrDefault**.

Syntax of a LINQ Query

In C#, LINQ queries can be written in two common styles: **query syntax** and **method syntax**. Both approaches are used to filter, sort, group, and select data from collections such as lists, arrays, and database results.

Query Syntax Overview

Query syntax provides a SQL-like way to write LINQ queries in C#. It is useful when you want the query to read step by step, especially for filtering, sorting, selecting, or grouping data from a collection.

- **from:** Identifies the data source and introduces a range variable, such as **from movie in movies**.
- **where:** Filters records based on a condition, such as selecting only movies from a specific genre.
- **orderby:** Sorts the results in ascending or descending order.
- **select:** Chooses the value or object that should be returned by the query.
- **group:** Organizes records into groups based on a selected property, such as genre or release year.

```
var result = from item in collection
              where item.Property == value
              orderby item.Property
              select item;

var sciFiMovies = from movie in movies
                  where movie.Genre == "Sci-Fi"
                  orderby movie.Rating descending
                  select movie;
```

Method Syntax

Method syntax uses LINQ extension methods with lambda expressions. It is compact and commonly used in C# applications.

```
var result = collection
    .Where(item => item.Property == value)
    .OrderBy(item => item.Property)
    .Select(item => item);

var sciFiMovies = movies
    .Where(movie => movie.Genre == "Sci-Fi")
```

```
.OrderByDescending(movie => movie.Rating)
.Select(movie => movie);
```

Common LINQ Clauses and Methods

- **Query syntax keywords:** **from** identifies the data source, **where** filters records, **orderby** sorts results, **select** chooses the returned value, and **group** groups records by a selected property.
- **Filtering and checking:** **Where()** filters records based on a condition, while **Any()** checks whether at least one record matches a condition.
- **Selecting and converting:** **Select()** chooses specific data from each item, and **ToList()** converts the LINQ result into a list.
- **Sorting:** **OrderBy()** sorts records in ascending order, and **OrderByDescending()** sorts records in descending order.
- **Calculating values:** **Average()** calculates the average value, **Max()** finds the highest value, and **Min()** finds the lowest value.
- **Finding records:** **First()** returns the first matching record and throws an error if none is found, while **FirstOrDefault()** returns the first matching record or **null** if no match exists.
- **Grouping and uniqueness:** **GroupBy()** groups records by a selected property, and **Distinct()** removes duplicate values from the result.
- **Paging:** **Skip()** skips a specified number of records, and **Take()** returns a specified number of records.
- **Output and iteration:** **Console.WriteLine()** prints output to the console, and a **foreach** loop repeats code for each item in a collection.

Quick tip: Use query syntax when you want SQL-like readability, and use method syntax when you want shorter code that works directly with lambda expressions.

- **Common use:** Together, they help write cleaner code for filtering employees, selecting specific fields, sorting records, and finding matching data.

Movie Model Class

The **Movie** class defines the structure of each movie object used in the LINQ examples. Each instance stores basic movie details such as the title, genre, release year, rating, and price.

- **Title:** A required movie name.
- **Genre:** An optional category such as Sci-Fi, Drama, or Thriller.
- **ReleaseYear:** The year the movie was released.
- **Rating:** A numeric score used for sorting and comparison.

- **Price:** The movie price used in filtering and ordering examples.

The following code defines the movie model used by the collection examples:

```
namespace MovieTutorial
{
    internal class Movie
    {
        public required string Title { get; set; }
        public string? Genre { get; set; }
        public int ReleaseYear { get; set; }
        public double Rating { get; set; }
        public int Price { get; set; }
    }
}
```

Movie Collection LINQ Examples

This example demonstrates how LINQ and lambda expressions can be used to query and analyze a collection of movies. The code creates a movie list, selects specific fields, filters results, sorts records, calculates summary values, groups items, and performs paging.

- **Selecting data:** Uses **Select** to extract movie titles and genres from the collection.
- **Filtering results:** Uses **Where** to find movies by genre, price, or other conditions.
- **Sorting records:** Uses **OrderBy** and **OrderByDescending** to organize movies by rating or price.
- **Calculating values:** Uses **Average**, **Max**, and **Min** to summarize movie ratings.
- **Finding records:** Uses **First** and **FirstOrDefault** to retrieve the best, worst, or cheapest movie.
- **Grouping and paging:** Uses **GroupBy**, **Skip**, and **Take** to organize and page through results.

The following code shows the complete LINQ workflow for the movie collection:

```
using System.Collections.Generic;

var movies = new List<MovieTutorial.Movie>
{
    new MovieTutorial.Movie { Title = "Inception", Genre = "Sci-Fi",
    ReleaseYear = 2010, Rating = 8.8, Price = 10 },
    new MovieTutorial.Movie { Title = "The Shawshank Redemption",
```

```

Genre = "Drama", ReleaseYear = 1994, Rating = 9.3, Price = 12 },
    new MovieTutorial.Movie { Title = "The Matrix", Genre = "Sci-Fi",
ReleaseYear = 1999, Rating = 8.7, Price = 9 },
    new MovieTutorial.Movie { Title = "Parasite", Genre = "Thriller",
ReleaseYear = 2019, Rating = 8.6, Price = 11 },
    new MovieTutorial.Movie { Title = "Interstellar", Genre = "Sci-
Fi", ReleaseYear = 2014, Rating = 8.6, Price = 13 }
};

// Select movie titles.
List<string> movieTitles = movies.Select(movie =>
movie.Title).ToList();
foreach (var title in movieTitles)
{
    Console.WriteLine(title);
}

// Filter movies by genre.
var sciFiMovies = movies.Where(movie => movie.Genre == "Sci-
Fi").ToList();
Console.WriteLine("\nSci-Fi Movies:");
foreach (var movie in sciFiMovies)
{
    Console.WriteLine(movie.Title);
}

// Check whether any recent movies exist.
var hasLatestMovies = movies.Any(movie => movie.ReleaseYear >= 2021);
Console.WriteLine($"Any Latest Movies: {hasLatestMovies}");

// Sort movies by rating.
Console.WriteLine("\nHigh Rated Movies:");
var sortedMoviesByRating = movies.OrderByDescending(movie =>
movie.Rating).ToList();
foreach (var movie in sortedMoviesByRating)
{
    Console.WriteLine($"{movie.Title} - Rating: {movie.Rating}");
}

```

```
Console.WriteLine("\nLow Rated Movies:");
var sortedMoviesByLowRating = movies.OrderBy(movie =>
movie.Rating).ToList();
foreach (var movie in sortedMoviesByLowRating)
{
    Console.WriteLine($"{movie.Title} - Rating: {movie.Rating}");
}

// Calculate rating statistics.
var averageRating = movies.Average(movie => movie.Rating);
Console.WriteLine($"{averageRating}");

var maxRating = movies.Max(movie => movie.Rating);
Console.WriteLine($"Max Rating of Movies: {maxRating}");

var bestRatedMovie = movies.First(movie => movie.Rating == maxRating);
Console.WriteLine($"Best Rated Movie: {bestRatedMovie?.Title} -
Rating: {bestRatedMovie?.Rating}");

var minRating = movies.Min(movie => movie.Rating);
Console.WriteLine($"Min Rating of Movies: {minRating}");

var worstRatedMovie = movies.FirstOrDefault(movie => movie.Rating ==
minRating);
Console.WriteLine($"Worst Rated Movie: {worstRatedMovie?.Title} -
Rating: {worstRatedMovie?.Rating}");

// Group movies by genre.
var groupedMoviesByGenre = movies.GroupBy(movie => movie.Genre);
foreach (var group in groupedMoviesByGenre)
{
    Console.WriteLine($"{group.Key}");
    foreach (var movie in group)
    {
        Console.WriteLine($"{movie.Title} - Rating: {movie.Rating}");
    }
}

// Filter, sort, and project budget thriller movies.
```

```
Console.WriteLine("\nBudget Thriller Movies (Price < 15):");
var budgetThrillerMovies = movies
    .Where(movie => movie.Genre == "Thriller" && movie.Price < 15)
    .OrderBy(movie => movie.Rating)
    .Select(movie => $"{movie.Title} - {movie.Rating}")
    .ToList();

foreach (var movie in budgetThrillerMovies)
{
    Console.WriteLine(movie);
}

// Page through movie results.
Console.WriteLine("\nPaged Movies (Skip 1, Take 3):");
var pagedMovies = movies.Skip(1).Take(3).ToList();
foreach (var movie in pagedMovies)
{
    Console.WriteLine(movie.Title);
}

// Query syntax example.
Console.WriteLine("\nThriller Movies By Query:");
var thrillerMoviesQuery = from movie in movies
    where movie.Genre == "Thriller"
    select movie;

foreach (var movie in thrillerMoviesQuery)
{
    Console.WriteLine(movie.Title);
}

// Find the cheapest movie.
var cheapestMovie = movies.OrderBy(movie => movie.Price).First();
Console.WriteLine($"Cheapest Movie: {cheapestMovie.Title}");

// List distinct genres.
Console.WriteLine("\nDistinct Genres:");
var distinctGenres = movies.Select(movie => movie.Genre).Distinct();
foreach (var genre in distinctGenres)
```

```
{  
    Console.WriteLine(genre);  
}
```

